

1 Semantica limbajului calculului cu propoziții

Considerăm mulțimea $\{T, F\}$, elementele ei având semnificațiile $T = \text{"adevărat"}$, respectiv $F = \text{"fals"}$. Pe mulțimea $\{T, F\}$ definim operațiile logice:

$$\begin{array}{lll} \neg & : & \{T, F\} \rightarrow \{T, F\} \quad (\text{negație}) \\ \wedge & : & \{T, F\}^2 \rightarrow \{T, F\} \quad (\text{conjuncție}) \\ \vee & : & \{T, F\}^2 \rightarrow \{T, F\} \quad (\text{disjuncție}) \\ \rightarrow & : & \{T, F\}^2 \rightarrow \{T, F\} \quad (\text{implicație}) \\ \leftrightarrow & : & \{T, F\}^2 \rightarrow \{T, F\} \quad (\text{echivalență}) \end{array}$$

prin tabelele:

<table><tr><td>$\neg$</td><td><table><tr><td>T</td><td>F</td></tr><tr><td>F</td><td>T</td></tr></table></td></tr></table> ,	$\neg$	<table><tr><td>T</td><td>F</td></tr><tr><td>F</td><td>T</td></tr></table>	T	F	F	T	<table><tr><td>$\wedge$</td><td>T</td><td>F</td></tr><tr><td>T</td><td>T</td><td>F</td></tr><tr><td>F</td><td>F</td><td>F</td></tr></table> ,	$\wedge$	T	F	T	T	F	F	F	F	<table><tr><td>$\vee$</td><td>T</td><td>F</td></tr><tr><td>T</td><td>T</td><td>T</td></tr><tr><td>F</td><td>T</td><td>F</td></tr></table> ,	$\vee$	T	F	T	T	T	F	T	F
$\neg$	<table><tr><td>T</td><td>F</td></tr><tr><td>F</td><td>T</td></tr></table>	T	F	F	T																					
T	F																									
F	T																									
$\wedge$	T	F																								
T	T	F																								
F	F	F																								
$\vee$	T	F																								
T	T	T																								
F	T	F																								
<table><tr><td>$\rightarrow$</td><td>T</td><td>F</td></tr><tr><td>T</td><td>T</td><td>F</td></tr><tr><td>F</td><td>T</td><td>T</td></tr></table> ,	$\rightarrow$	T	F	T	T	F	F	T	T	<table><tr><td>$\leftrightarrow$</td><td>T</td><td>F</td></tr><tr><td>T</td><td>T</td><td>F</td></tr><tr><td>F</td><td>F</td><td>T</td></tr></table> .	$\leftrightarrow$	T	F	T	T	F	F	F	T							
$\rightarrow$	T	F																								
T	T	F																								
F	T	T																								
$\leftrightarrow$	T	F																								
T	T	F																								
F	F	T																								

Numim funcție de adevăr orice funcție $h : V \rightarrow \{T, F\}$.

Lema 1.7.1. Fie h o funcție de adevăr. Atunci există și este unică

$I(h) : FORM \rightarrow \{T, F\}$, astfel încât următoarele cerințe să fie îndeplinite:

1. pentru orice $\alpha \in V$, $I(h)(\alpha) = h(\alpha)$;
2. pentru orice $\alpha, \beta \in FORM$,
 - 2.1 $I(h)(\neg\alpha) = \neg I(h)(\alpha)$;
 - 2.2 $I(h)(\alpha \wedge \beta) = I(h)(\alpha) \wedge I(h)(\beta)$;
 - 2.3 $I(h)(\alpha \vee \beta) = I(h)(\alpha) \vee I(h)(\beta)$;
 - 2.4 $I(h)(\alpha \rightarrow \beta) = I(h)(\alpha) \rightarrow I(h)(\beta)$;
 - 2.5 $I(h)(\alpha \leftrightarrow \beta) = I(h)(\alpha) \leftrightarrow I(h)(\beta)$.

Definiția 1.7.4. Spunem că formulele α, β sunt semantic echivalente (notat $\alpha \equiv \beta$), dacă $\aleph(\alpha) = \aleph(\beta)$.

Observație Relația de echivalență semantică este o relație de echivalență pe mulțimea $FORM$.

Lema 1.7.4. Pentru orice $\alpha, \beta, \gamma \in FORM$,

1. $(\alpha \vee \alpha) \equiv \alpha, (\alpha \wedge \alpha) \equiv \alpha,$
2. $(\neg(\neg\alpha)) \equiv \alpha,$
3. $(\alpha \rightarrow \beta) \equiv ((\neg\alpha) \vee \beta), (\alpha \leftrightarrow \beta) \equiv ((\alpha \rightarrow \beta) \wedge (\beta \rightarrow \alpha)),$
4. $(\neg(\alpha \wedge \beta)) \equiv ((\neg\alpha) \vee (\neg\beta)), (\neg(\alpha \vee \beta)) \equiv ((\neg\alpha) \wedge (\neg\beta)),$
5. $(\alpha \wedge (\beta \vee \gamma)) \equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma)),$
 $(\alpha \vee (\beta \wedge \gamma)) \equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma)),$
6. $(\alpha \vee \beta) \equiv ((\neg\alpha) \rightarrow \beta), (\alpha \wedge \beta) \equiv (\neg(\alpha \rightarrow (\neg\beta))).$

Demonstrație

Justificarea este imediată și rezultă direct pe baza definițiilor.

Observație Din proprietățile (3) și (6) rezultă că mulțimea de conective $\{\rightarrow, \neg\}$ este completă din punct de vedere semantic, în sensul că pentru orice formulă α există formula α' , astfel încât $\alpha \equiv \alpha'$ și singurele simboluri de tip conectivă din structura simbolică α' sunt $\{\rightarrow, \neg\}$.

Lema 1.7.5 Dacă $\alpha \equiv \beta$ și $\gamma \equiv \delta$, atunci

1. $(\neg\alpha) \equiv (\neg\beta),$
2. $(\alpha\rho\gamma) \equiv (\beta\rho\delta)$ pentru orice $\rho \in L \setminus \{\rho\}.$

Demonstrație Justificarea este imediată și rezultă direct pe baza definițiilor.

Teorema 1.7.1 (Teorema de consistență a calculului cu propoziții)

Orice teoremă este tautologie.

Demonstrație Justificarea afirmației rezultă imediat prin inducție asupra lungimii demonstrației formale, dacă demonstrăm că orice exemplu de axiomă este tautologie.

Fie $\alpha, \beta, \gamma \in FORM$ arbitrare, $\sigma = \{\alpha \mid a, \beta \mid b, \gamma \mid c\}$. Pentru $I \in \mathfrak{S}$ arbitrară, utilizând observațiile precedente, obținem

1. Axioma $\overline{\alpha_1}$:

$$\begin{aligned} I(\overline{\alpha_1}\sigma) &= I((\alpha \rightarrow (\beta \rightarrow \alpha))) = I(\alpha) \rightarrow (I(\beta) \rightarrow I(\alpha)) \\ &= \neg I(\alpha) \vee \neg I(\beta) \vee I(\alpha) = T \end{aligned}$$

2. Axioma $\overline{\alpha_2}$:

$$\begin{aligned}
I(\overline{\alpha_2}\sigma) &= I(((\alpha \rightarrow (\alpha \rightarrow \beta)) \rightarrow (\alpha \rightarrow \beta))) \\
&= \neg I((\alpha \rightarrow (\alpha \rightarrow \beta)) \vee I(\alpha \rightarrow \beta)) \\
&= \neg(\neg I(\alpha) \vee \neg I(\alpha) \vee I(\beta)) \vee (\neg I(\alpha) \vee I(\beta)) \\
&= \neg(\neg I(\alpha) \vee I(\beta)) \vee (\neg I(\alpha) \vee I(\beta)) \\
&= (I(\alpha) \wedge \neg I(\beta)) \vee (\neg I(\alpha) \vee I(\beta)) \\
&= (I(\alpha) \vee \neg I(\alpha) \vee I(\beta)) \wedge (\neg I(\beta) \vee \neg I(\alpha) \vee I(\beta)) = T.
\end{aligned}$$

3. Axioma $\overline{\alpha_3}$:

$$\begin{aligned}
I(\overline{\alpha_3}\sigma) &= I(((\alpha \rightarrow \beta) \rightarrow ((\beta \rightarrow \gamma) \rightarrow (\alpha \rightarrow \gamma)))) \\
&= \neg I((\alpha \rightarrow \beta) \vee (\neg I((\beta \rightarrow \gamma)) \vee I((\alpha \rightarrow \gamma)))) \\
&= \neg(\neg I(\alpha) \vee I(\beta)) \vee (\neg(\neg I(\beta) \vee I(\gamma)) \vee (\neg I(\alpha) \vee I(\gamma))) \\
&= (I(\alpha) \wedge \neg I(\beta)) \vee ((I(\beta) \wedge \neg I(\gamma)) \vee (\neg I(\alpha) \vee I(\gamma))) \\
&= (I(\alpha) \wedge \neg I(\beta)) \vee ((I(\beta) \vee \neg I(\alpha) \vee I(\gamma)) \wedge \\
&\quad (\neg I(\gamma) \vee \neg I(\alpha) \vee I(\gamma))) \\
&= (I(\alpha) \wedge \neg I(\beta)) \vee ((I(\beta) \vee \neg I(\alpha) \vee I(\gamma)) \wedge T) \\
&= (I(\alpha) \wedge \neg I(\beta)) \vee (I(\beta) \vee \neg I(\alpha) \vee I(\gamma)) \\
&= (I(\alpha) \vee I(\beta) \vee \neg I(\alpha) \vee I(\gamma)) \wedge \\
&\quad (\neg I(\beta) \vee I(\beta) \vee \neg I(\alpha) \vee I(\gamma)) \\
&= T \wedge T = T
\end{aligned}$$

4. Axioma $\overline{\alpha_4}$:

$$\begin{aligned}
I(\overline{\alpha_4}\sigma) &= I(((\alpha \leftrightarrow \beta) \rightarrow (\alpha \rightarrow \beta))) \\
&= \neg I((\alpha \leftrightarrow \beta) \vee I((\alpha \rightarrow \beta))) \\
&= \neg(I((\alpha \rightarrow \beta)) \wedge I((\beta \rightarrow \alpha))) \vee I((\alpha \rightarrow \beta)) \\
&= \neg I((\alpha \rightarrow \beta)) \vee \neg I((\beta \rightarrow \alpha)) \vee I((\alpha \rightarrow \beta)) = T
\end{aligned}$$

5. Axioma $\overline{\alpha_5}$:

$$\begin{aligned}
I(\overline{\alpha_5}\sigma) &= I(((\alpha \leftrightarrow \beta) \rightarrow (\beta \rightarrow \alpha))) \\
&= \neg I((\alpha \leftrightarrow \beta) \vee I((\beta \rightarrow \alpha))) \\
&= \neg(I((\alpha \rightarrow \beta)) \wedge I((\beta \rightarrow \alpha))) \vee I((\beta \rightarrow \alpha)) \\
&= \neg I((\alpha \rightarrow \beta)) \vee \neg I((\beta \rightarrow \alpha)) \vee I((\beta \rightarrow \alpha)) = T
\end{aligned}$$

6. Axioma $\overline{\alpha_6}$:

$$\begin{aligned}
I(\overline{\alpha_6}\sigma) &= I(((\alpha \rightarrow \beta) \rightarrow ((\beta \rightarrow \alpha) \rightarrow (\alpha \leftrightarrow \beta)))) \\
&= \neg I((\alpha \rightarrow \beta)) \vee (\neg I((\beta \rightarrow \alpha)) \vee I((\alpha \leftrightarrow \beta))) \\
&= \neg I((\alpha \rightarrow \beta)) \vee (\neg I((\beta \rightarrow \alpha)) \vee \\
&\quad (I((\alpha \rightarrow \beta)) \wedge I((\beta \rightarrow \alpha)))) \\
&= \neg I((\alpha \rightarrow \beta)) \vee ((\neg I((\beta \rightarrow \alpha)) \vee I((\alpha \rightarrow \beta))) \wedge \\
&\quad (\neg I((\beta \rightarrow \alpha)) \vee I((\beta \rightarrow \alpha)))) \\
&= \neg I((\alpha \rightarrow \beta)) \vee ((\neg I((\beta \rightarrow \alpha)) \vee I((\alpha \rightarrow \beta))) \wedge T) \\
&= \neg I((\alpha \rightarrow \beta)) \vee (\neg I((\beta \rightarrow \alpha)) \vee I((\alpha \rightarrow \beta))) = T
\end{aligned}$$

7. Axioma $\overline{\alpha_7}$:

$$\begin{aligned}
I(\overline{\alpha_7}\sigma) &= I((((\neg \alpha) \rightarrow (\neg \beta)) \rightarrow (\beta \rightarrow \alpha))) \\
&= \neg I(((\neg \alpha) \rightarrow (\neg \beta))) \vee I((\beta \rightarrow \alpha)) \\
&= \neg (\neg I((\neg \alpha)) \vee I((\neg \beta))) \vee (\neg I(\beta) \vee I(\alpha)) \\
&= \neg (I(\alpha) \vee \neg I(\beta)) \vee (\neg I(\beta) \vee I(\alpha)) \\
&= (\neg I(\alpha) \wedge I(\beta)) \vee (\neg I(\beta) \vee I(\alpha)) \\
&= (\neg I(\alpha) \vee \neg I(\beta) \vee I(\alpha)) \wedge (I(\beta) \vee \neg I(\beta) \vee I(\alpha)) \\
&= T \wedge T = T
\end{aligned}$$

8. Axioma $\overline{\alpha_8}$:

$$\begin{aligned}
I(\overline{\alpha_8}\sigma) &= I(((\alpha \vee \beta) \leftrightarrow ((\neg \alpha) \rightarrow \beta))) \\
&= I(((\alpha \vee \beta) \rightarrow ((\neg \alpha) \rightarrow \beta))) \wedge \\
&\quad \wedge I((((\neg \alpha) \rightarrow \beta) \rightarrow (\alpha \vee \beta))) \\
&= (\neg I((\alpha \vee \beta)) \vee I(((\neg \alpha) \rightarrow \beta))) \wedge \\
&\quad \wedge (\neg I(((\neg \alpha) \rightarrow \beta)) \vee I((\alpha \vee \beta))) \\
&= (\neg I((\alpha \vee \beta)) \vee (\neg I((\neg \alpha)) \vee I(\beta))) \wedge \\
&\quad \wedge (\neg (\neg I((\neg \alpha)) \vee I(\beta)) \vee I((\alpha \vee \beta))) \\
&= (\neg I((\alpha \vee \beta)) \vee (I(\alpha) \vee I(\beta))) \wedge \\
&\quad \wedge (\neg (I(\alpha) \vee I(\beta)) \vee I((\alpha \vee \beta))) \\
&= (\neg I((\alpha \vee \beta)) \vee I((\alpha \vee \beta))) \wedge \\
&\quad \wedge (\neg I((\alpha \vee \beta)) \vee I((\alpha \vee \beta))) \\
&= T
\end{aligned}$$

9. Axioma $\overline{\alpha_9}$:

$$\begin{aligned}
I(\overline{\alpha_9}\sigma) &= I(((\alpha \wedge \beta) \leftrightarrow (\neg(\neg\alpha) \vee (\neg\beta)))) \\
&= I(((\alpha \wedge \beta) \rightarrow (\neg(\neg\alpha) \vee (\neg\beta)))) \wedge \\
&\quad I(((\neg(\neg\alpha) \vee (\neg\beta)) \rightarrow (\alpha \wedge \beta))) \\
&= (\neg I((\alpha \wedge \beta)) \vee I((\neg(\neg\alpha) \vee (\neg\beta)))) \wedge \\
&\quad (\neg I((\neg(\neg\alpha) \vee (\neg\beta))) \vee I((\alpha \wedge \beta))) \\
&= (\neg I((\alpha \wedge \beta)) \vee \neg(I((\neg\alpha)) \vee I((\neg\beta)))) \wedge \\
&\quad ((I((\neg\alpha)) \vee I((\neg\beta))) \vee I((\alpha \wedge \beta))) \\
&= (\neg I((\alpha \wedge \beta)) \vee \neg(\neg I(\alpha) \vee \neg I(\beta))) \wedge \\
&\quad ((\neg I(\alpha) \vee \neg I(\beta)) \vee I((\alpha \wedge \beta))) \\
&= (\neg I((\alpha \wedge \beta)) \vee \neg\neg(I(\alpha) \wedge I(\beta))) \wedge \\
&\quad (\neg(I(\alpha) \wedge I(\beta)) \vee I((\alpha \wedge \beta))) \\
&= (\neg I((\alpha \wedge \beta)) \vee I((\alpha \wedge \beta))) \wedge \\
&\quad (\neg I((\alpha \wedge \beta)) \vee I((\alpha \wedge \beta))) \\
&= T \wedge T = T
\end{aligned}$$

În concluzie, orice exemplu de axiomă este tautologie.
Să observăm că pentru orice formule α, β ,

$$\aleph(\alpha) \cap \aleph((\alpha \rightarrow \beta)) \subset \aleph(\beta).$$

Într-adevăr, dacă $I \in \aleph(\alpha) \cap \aleph((\alpha \rightarrow \beta))$, atunci $I(((\alpha \rightarrow \beta))) = I(\alpha) = T$, și

$$\begin{aligned}
I(((\alpha \rightarrow \beta))) &= I(((\neg\alpha) \vee \beta)) = \neg I(\alpha) \vee I(\beta) \\
&= F \vee I(\beta) = I(\beta),
\end{aligned}$$

deci $I \in \aleph(\beta)$.

Demonstrăm prin inducție asupra lungimii demonstrației formale că orice $\alpha \in T_h$ este tautologie.

Dacă admite o demonstrație formală de lungime 1, atunci α este exemplu de axiomă, deci α este tautologie. Presupunem că orice formulă demonstrabilă ce admite o demonstrație formală de lungime mai mică sau egală cu n este tautologie.

Fie $\alpha_1, \dots, \alpha_n, \alpha_{n+1} = \alpha$ demonstrație formală. Evident, pentru orice k , $1 \leq k \leq n$, $\alpha_1, \dots, \alpha_k$ este demonstrație formală, deci, conform ipotezei inductive, formulele α_k , $k = 1, \dots, n$ sunt tautologii.

5 Verificarea automată a validabilității formulelor

Fie H, Γ mulțimi de formule. Procedura $\wp$ este o procedură de demonstrare automată dacă decide $H \vdash \Gamma$ cu alternativa $H \not\vdash \Gamma$. Convenim să numim soluție calculată de procedura de demonstrare automată $\wp$ secvența de etape intermediare ale evoluției determinate de procedură. În aplicații de interes practic, mulțimile H, Γ sunt finite, caz în care rezultatele stabilite în cadrul secțiunilor precedente permit reducerea problemei verificării, dacă $H \vdash \Gamma$ la verificarea

validabilității/invalidabilității unei singure formule. Într-adevăr, dacă $H = \{\alpha_1, \dots, \alpha_n\}, \Gamma = \{\beta_1, \dots, \beta_m\}$, atunci conform rezultatului stabilit de *Teorema* 1.5.3., $H \vdash \Gamma$, dacă și numai dacă $\left(\bigwedge_{i=1}^n \alpha_i \rightarrow \bigvee_{j=1}^m \beta_j \right)$ este tautologie. De asemenea, o caracterizare echivalentă

este $H \vdash \Gamma$, dacă și numai dacă $\left(\bigwedge_{i=1}^n \alpha_i \right) \wedge \left(\bigwedge_{j=1}^m (\neg \beta_j) \right)$ este invalidabilă.

În consecință, procedurile de demonstrare automată pot fi gândite ca proceduri pentru verificarea validabilității/invalidabilității unei singure formule. În esență, o procedură de demonstrare automată este o metodă de căutare sistematică a unui model pentru o formulă dată. O tehnică de demonstrare automată $\wp$ care verifică $\{\alpha_1, \dots, \alpha_n\} \vdash \{\beta_1, \dots, \beta_m\}$ procedând la verificarea invalidabilității formulei $\left(\bigwedge_{i=1}^n \alpha_i \right) \wedge \left(\bigwedge_{j=1}^m (\neg \beta_j) \right)$ se numește metodă de respingere.

O procedură de verificare a validabilității/invalidabilității unei formule date α este imediată și anume, revine la generarea a 2^n funcții de adevăr, unde n este numărul propozițiilor elementare care apar în structura simbolică α . Dacă valoarea calculată pentru α este T pentru toate interpretările induse de funcțiile de adevăr astfel generate, atunci α este tautologie. Dacă cel puțin o astfel de interpretare calculează T , atunci α este validabilă, respectiv dacă în toate cele 2^n interpretări valoarea calculată este F , atunci α este invalidabilă.

Exemplul 1.9.1. Fie $a, b \in V, \alpha = (a \rightarrow (b \rightarrow (a \wedge b)))$. Deoarece $n = 2$

vor fi generate 2^2 funcții de adevăr.

$\hbar(a)$	$\hbar(b)$	$I(\hbar)(a \wedge b)$	$I(\hbar)\left(\begin{smallmatrix} b \rightarrow \\ (a \wedge b) \end{smallmatrix}\right)$	$I(\hbar)\left(\begin{smallmatrix} a \rightarrow \\ (b \rightarrow (a \wedge b)) \end{smallmatrix}\right)$
T	T	T	T	T
T	F	F	T	T
F	T	F	F	T
F	F	F	T	T

Rezultă $\alpha = (a \rightarrow (b \rightarrow (a \wedge b)))$ este tautologie, deci $\alpha \in T_h$.

5.1 1.9.1. Normalizarea CNF a formulelor limbajului calculului cu propoziții.

Definiția 1.9.1.1. Un literal este o propoziție elementară sau negația unei propoziții elementare. Mulțimea literalilor este notată $V \cup \neg V$.

Dacă $p \in V$ atunci literalii $p, (\neg p)$ se numesc literalii complementari. Literalii din V se numesc literalii pozitivi respectiv literalii din $\neg V$ se numesc literalii negativi.

Definiția 1.9.1.2. Se numește clauză o structură simbolică $k = \bigvee_{i=1}^n L_i$ de tip disjunctiv pentru o mulțime de literalii $\{L_1, \dots, L_n\}$. Clauza corespunzătoare mulțimii vide de literalii se numește clauza vidă și este notată $\square$.

Observație Pentru $k \neq \square$, mulțimea modelelor clauzei $k = \bigvee_{i=1}^n L_i; n \geq 1$ este evident $\aleph(k) = \bigcup_{i=1}^n \aleph(L_i)$. Conform convențiilor Bourbaki, calculul mulțimii $\aleph(\square)$ revine la reuniune după mulțimea vidă de indici, deci $\aleph(\square) = \emptyset$. Cu alte cuvinte, clauza vidă este invalidabilă.

Definiția 1.9.1.3. Se numește formulă normalizată CNF (Conjunctive Normal Form) o structură simbolică de tip conjunctiv $\alpha = \bigwedge_{i=1}^n k_i$, unde $\{k_1, \dots, k_n\}$ este o mulțime de clauze. Mulțimea clauzelor

$S(\alpha) = \{k_1, \dots, k_n\}$ se numește reprezentare clauzală pentru formula α . Formula CNF corespunzătoare mulțimii vide de clauze se numește formula vidă și este notată $\emptyset$.

Observație Dacă $\alpha = \bigwedge_{i=1}^n k_i; n \geq 1$, atunci $\aleph(\alpha) = \aleph(S(\alpha)) = \bigcap_{i=1}^n \aleph(k_i)$. Conform convențiilor Bourbaki, calculul mulțimii $\aleph(\emptyset)$ revine la intersecție după mulțime vidă de indici, deci $\aleph(\emptyset) = \mathfrak{S}$. Cu alte cuvinte, formula vidă este tautologie.

Rezultatul stabilit de următoarea teoremă exprimă faptul că, din punct de vedere semantic, se poate presupune întotdeauna că se dispune de reprezentări normalizate CNF pentru formulele limbajului calculului cu propoziții.

Definiția 1.9.1.4. Fie $\alpha, \beta \in FORM$. Spunem că β este o subformulă a formulei α , dacă β este o componentă a unui SGF de lungime minimă pentru α .

Exemplul 1.9.1.1. Fie $a, b \in V$, $\alpha = (a \rightarrow (b \rightarrow (a \wedge b)))$.

Evident, secvența

$$a, b, (a \wedge b), (b \rightarrow (a \wedge b)), (a \rightarrow (b \rightarrow (a \wedge b))) = \alpha$$

este un SGF de lungime minimă pentru α .

Rezultă că mulțimea subformulelor lui α este

$$\{a, b, (a \wedge b), (b \rightarrow (a \wedge b)), (a \rightarrow (b \rightarrow (a \wedge b)))\}.$$

Observație Orice subformulă a unei formule α este o formulă. Formula β este subformulă a formulei α , dacă și numai dacă β este o componentă a oricărui SGF pentru α .

Teorema 1.9.1.1. Pentru orice $\alpha \in FORM$ există α' formulă normalizată CNF și $\alpha \equiv \alpha'$.

Demonstrație Aplicăm succesiv subformulelor formulei α următoarele transformări:

T1. Eliminarea conectivei “ $\leftrightarrow$ ”: Fiecare subformulă de tipul $(\beta \leftrightarrow \gamma)$ se substituie cu $((\beta \rightarrow \gamma) \wedge (\gamma \rightarrow \beta))$;

$$(\beta \leftrightarrow \gamma) \equiv ((\beta \rightarrow \gamma) \wedge (\gamma \rightarrow \beta)).$$

T2. Eliminarea conectivei “ $\rightarrow$ ”: Fiecare subformulă de tipul $(\beta \rightarrow \gamma)$ se substituie cu $((\neg\beta) \vee \gamma)$;

$$(\beta \rightarrow \gamma) \equiv ((\neg\beta) \vee \gamma).$$

T3. Se aduc negațiile în fața literalilor:

ι) Fiecare subformulă de tipul $(\neg(\beta \vee \gamma))$ se substituie cu $((\neg\beta) \wedge (\neg\gamma))$;

$$(\neg(\beta \vee \gamma)) \equiv ((\neg\beta) \wedge (\neg\gamma)).$$

ι) Fiecare subformulă de tipul $(\neg(\beta \wedge \gamma))$ se substituie cu $((\neg\beta) \vee (\neg\gamma))$;

$$(\neg(\beta \wedge \gamma)) \equiv ((\neg\beta) \vee (\neg\gamma)).$$

T4. Eliminarea negațiilor multiple: Fiecare subformulă de tipul $(\neg(\neg\beta))$ se substituie cu β ;

$$(\neg(\neg\beta)) \equiv \beta.$$

T5. Obținerea structurii CNF: Fiecare subformulă de tipul $(\beta \vee (\gamma \wedge \delta))$ se substituie prin $((\beta \vee \gamma) \wedge (\beta \vee \delta))$;

$$(\beta \vee (\gamma \wedge \delta)) \equiv ((\beta \vee \gamma) \wedge (\beta \vee \delta)),$$

respectiv $((\gamma \wedge \delta) \vee \beta)$ se substituie prin $((\gamma \vee \beta) \wedge (\delta \vee \beta))$;

$$((\gamma \wedge \delta) \vee \beta) \equiv ((\gamma \vee \beta) \wedge (\delta \vee \beta)).$$

Deoarece aplicarea fiecărei transformări determină substituirea unei subformule cu o subformulă semantic echivalentă cu ea, la fiecare etapă se obține o formulă semantic echivalentă cu formula inițială. Rezultă în final o reprezentare normalizată *CNF* semantic echivalentă cu formula inițială.

Exemplul 1.9.1.2. Fie $a, b \in V$, $\alpha = (((\neg b) \rightarrow (\neg a)) \longleftrightarrow (a \rightarrow b))$.

Mulțimea subformulelor formulei α este

$$F_0 = \{a, b, (\neg a), (\neg b), (a \rightarrow b), (((\neg b) \rightarrow (\neg a)) \longleftrightarrow (a \rightarrow b))\}.$$

Aplicarea transformării *T1* determină substituirea subformulei

$$(((\neg b) \rightarrow (\neg a)) \longleftrightarrow (a \rightarrow b))$$

prin

$$((((\neg b) \rightarrow (\neg a)) \rightarrow (a \rightarrow b)) \wedge ((a \rightarrow b) \rightarrow ((\neg b) \rightarrow (\neg a))))).$$

Rezultă

$$\alpha_1 = ((((\neg b) \rightarrow (\neg a)) \rightarrow (a \rightarrow b)) \wedge ((a \rightarrow b) \rightarrow ((\neg b) \rightarrow (\neg a))))).$$

Aplicarea transformării *T2* determină substituirea subformulelor:

$$\begin{array}{ll} (((\neg b) \rightarrow (\neg a)) \rightarrow (a \rightarrow b)) & \text{prin } ((\neg((\neg b) \rightarrow (\neg a))) \vee (a \rightarrow b)) \\ ((a \rightarrow b) \rightarrow ((\neg b) \rightarrow (\neg a))) & \text{prin } ((\neg(a \rightarrow b)) \vee ((\neg b) \rightarrow (\neg a))) \\ (a \rightarrow b) & \text{prin } ((\neg a) \vee b) \\ ((\neg b) \rightarrow (\neg a)) & \text{prin } ((\neg(\neg b)) \vee (\neg a)). \end{array}$$

Rezultă

$$\alpha_2 = \left(\begin{array}{l} ((\neg(\neg(\neg b)) \vee (\neg a))) \vee ((\neg a) \vee b) \wedge \\ \wedge ((\neg(\neg(\neg a) \vee b)) \vee ((\neg(\neg b)) \vee (\neg a))) \end{array} \right).$$

Aplicarea transformării $T3$ determină substituirea subformulelor:

$$\begin{array}{ccc} (\neg((\neg(\neg b)) \vee (\neg a))) & \text{prin} & ((\neg(\neg(\neg b))) \wedge (\neg(\neg a))) \\ (\neg((\neg a) \vee b)) & \text{prin} & ((\neg(\neg a)) \wedge (\neg b)). \end{array}$$

Rezultă

$$\alpha_3 = \left(\begin{array}{c} (((\neg(\neg(\neg b))) \wedge (\neg(\neg a))) \vee ((\neg a) \vee b)) \wedge \\ \wedge (((\neg(\neg a)) \wedge (\neg b)) \vee ((\neg(\neg b)) \vee (\neg a))) \end{array} \right).$$

Aplicarea transformării $T4$ determină substituirea subformulelor:

$$\begin{array}{ccc} (\neg(\neg b)) & \text{prin} & b \\ (\neg(\neg a)) & \text{prin} & a. \end{array}$$

Rezultă

$$\alpha_4 = (((\neg b) \wedge a) \vee ((\neg a) \vee b)) \wedge ((a \wedge (\neg b)) \vee (b \vee (\neg a))).$$

Aplicarea transformării $T5$ determină substituirea subformulelor:

$$\begin{array}{ccc} (((\neg b) \wedge a) \vee ((\neg a) \vee b)) & \text{prin} & (((\neg b) \vee ((\neg a) \vee b)) \wedge (a \vee ((\neg a) \vee b))) \\ ((a \wedge (\neg b)) \vee (b \vee (\neg a))) & \text{prin} & ((a \vee (b \vee (\neg a))) \wedge ((\neg b) \vee (b \vee (\neg a)))) \end{array}.$$

Rezultă reprezentarea normalizată CNF pentru formula α ,

$$\alpha' = \left(\begin{array}{c} (((\neg b) \vee ((\neg a) \vee b)) \wedge (a \vee ((\neg a) \vee b))) \wedge \\ \wedge ((a \vee (b \vee (\neg a))) \wedge ((\neg b) \vee (b \vee (\neg a)))) \end{array} \right)$$

și $\alpha \equiv \alpha'$.

Reprezentarea clauzală este

$$S(\alpha) = \left\{ \begin{array}{l} k_1 = ((\neg b) \vee ((\neg a) \vee b)), \quad k_2 = (a \vee ((\neg a) \vee b)), \\ k_3 = (a \vee (b \vee (\neg a))), \quad k_4 = ((\neg b) \vee (b \vee (\neg a))) \end{array} \right\}.$$

Exemplul 1.9.1.3. Fie $m, p, o, c, i \in V, \beta = ((m \wedge p) \rightarrow i)$

$$H = \{\alpha_1 = (p \rightarrow (o \wedge (\neg c))), \alpha_2 = ((m \wedge (\neg c)) \rightarrow i)\}.$$

Conform rezultatelor stabilite în secțiunile precedente, verificarea deductibilității formulei β sub familia de ipoteze H , corespunde verificării proprietății de a fi tautologie a formulei $((\alpha_1 \wedge \alpha_2) \rightarrow \beta)$ sau echivalent la demonstrarea invalidabilității formulei

$$\gamma = ((\alpha_1 \wedge \alpha_2) \wedge (\neg \beta)).$$

Deoarece numărul propozițiilor elementare $n = 5$, verificările pot fi efectuate după modelul din *Exemplul 1.9.1.* pe baza generării a 2^5 funcții de adevăr.

Dată fiind structura de tip conjunctiv, o reprezentare normalizată CNF a formulei γ este $\gamma' = ((\alpha'_1 \wedge \alpha'_2) \wedge (\neg\beta)')$, unde $\alpha'_1, \alpha'_2, (\neg\beta)'$ sunt reprezentări normalizate CNF corespunzătoare respectiv formulelor $\alpha_1, \alpha_2, (\neg\beta)$. De asemenea, $S(\gamma) = S(\alpha_1) \cup S(\alpha_2) \cup S((\neg\beta))$ este o reprezentare clauzală pentru γ , unde $S(\alpha_1), S(\alpha_2), S((\neg\beta))$ sunt reprezentări clauzale respectiv pentru $\alpha_1, \alpha_2, (\neg\beta)$.

Aplicând construcția descrisă în demonstrația *Teoremei 1.9.1.1.*, rezultă

$$\begin{aligned}\alpha_1 &= (p \rightarrow (o \wedge (\neg c))) \equiv ((\neg p) \vee (o \wedge (\neg c))) \equiv \\ &\equiv (((\neg p) \vee o) \wedge ((\neg p) \vee (\neg c)))\end{aligned}$$

deci

$$S(\alpha_1) = \{k_1 = ((\neg p) \vee o), k_2 = ((\neg p) \vee (\neg c))\}.$$

$$\begin{aligned}\alpha_2 &= ((m \wedge (\neg c)) \rightarrow i) \equiv ((\neg(m \wedge (\neg c))) \vee i) \equiv \\ &\equiv (((\neg m) \vee (\neg(\neg c))) \vee i) \equiv (((\neg m) \vee c) \vee i)\end{aligned}$$

deci

$$S(\alpha_2) = \{k_3 = (((\neg m) \vee c) \vee i)\}.$$

$$\begin{aligned}(\neg\beta) &= (\neg((m \wedge p) \rightarrow i)) \equiv (\neg(m \wedge p)) (\neg((\neg(m \wedge p)) \vee i)) \equiv \\ &\equiv (\neg(\neg(m \wedge p))) \wedge (\neg i) \equiv ((m \wedge p) \wedge (\neg i))\end{aligned}$$

deci

$$S((\neg\beta)) = \{k_4 = m, k_5 = p, k_6 = (\neg i)\}.$$

Pentru formula γ rezultă reprezentarea clauzală

$$S(\gamma) = \{k_1, k_2, k_3, k_4, k_5, k_6\}.$$

Observație Deoarece pentru orice formulă α , $(\alpha \wedge \alpha) \equiv \alpha$, $(\alpha \vee \alpha) \equiv \alpha$, obținem următoarele reguli de simplificare:

1. Prin eliminarea duplicatelor literalilor din clauza k rezultă clauza k' și $\aleph(k) = \aleph(k')$.

2. Prin eliminarea duplicatelor clauzelor într-o reprezentare clauzală $S(\alpha)$ rezultă reprezentarea clauzală $S'(\alpha)$ și

$$\aleph(\alpha) = \aleph(S(\alpha)) = \aleph(S'(\alpha)).$$

De asemenea, deoarece pentru orice formule α, β , $(\alpha \wedge \beta) \equiv (\beta \wedge \alpha)$, $(\alpha \vee \beta) \equiv (\beta \vee \alpha)$, rezultă

$$\begin{aligned}
\varphi(n_1) &= \{((a \rightarrow b) \vee ((\neg a) \rightarrow c))\} \Rightarrow \{(b \vee c)\} \\
\varphi(n_2) &= \{(a \rightarrow b)\} \Rightarrow \{(b \vee c)\} \\
\varphi(n_3) &= \{((\neg a) \rightarrow c)\} \Rightarrow \{(b \vee c)\} \\
\varphi(n_4) &= \{b\} \Rightarrow \{(b \vee c)\} \\
\varphi(n_5) &= \emptyset \Rightarrow \{a, (b \vee c)\} \\
\varphi(n_6) &= \{b\} \Rightarrow \{b, c\} \text{ (secvent axiomă)} \\
\varphi(n_7) &= \emptyset \Rightarrow \{a, b, c\} \\
\varphi(n_8) &= \{c\} \Rightarrow \{(b \vee c)\} \\
\varphi(n_9) &= \emptyset \Rightarrow \{(\neg a), (b \vee c)\} \\
\varphi(n_{10}) &= \{a\} \Rightarrow \{(b \vee c)\} \\
\varphi(n_{11}) &= \{a\} \Rightarrow \{b, c\} \\
\varphi(n_{12}) &= \{c\} \Rightarrow \{b, c\} \text{ (secvent axiomă)}.
\end{aligned}$$

Mulțimea frontieră este $\partial T = \{n_6, n_7, n_{11}, n_{12}\}$, deci

$S(\alpha) = \{k_1, k_2, k_3, k_4\}$, $S'(\alpha) = \{k_2, k_3\}$ unde,

$k_1 = (\neg b) \vee b \vee c$ (clauză tautologie) – asociată vârfului n_6

$k_2 = a \vee b \vee c$ – asociată vârfului n_7

$k_3 = (\neg a) \vee b \vee c$ – asociată vârfului n_{11}

$k_4 = (\neg c) \vee b \vee c$ (clauză tautologie) – asociată vârfului n_{12} .

Cu alte cuvinte, $\alpha = (((a \rightarrow b) \vee ((\neg a) \rightarrow c)) \rightarrow (b \vee c)) \equiv \alpha'$, unde

$\alpha' = (((a \vee b) \vee c) \wedge (((\neg a) \vee b) \vee c))$ este formă normală conjunctivă.

Rezultă $\aleph(\alpha) = \aleph(((a \vee b) \vee c)) \cap \aleph(((\neg a) \vee b) \vee c) = \aleph((b \vee c))$.

5.2 1.9.2 Metoda Davis-Putnam pentru verificarea validabilității formulelor limbajului calculului cu propoziții.

Demonstratorul de teoreme Davis-Putnam este un sistem de verificare a validabilității formulelor bazat pe tehnica de respingere. Prelucrarea inițiată de metoda Davis-Putnam este efectuată asupra reprezentării clauzale corespunzătoare formulei testate.

Dacă α este o structură simbolică și λ un simbol, convenim să notăm $\alpha \langle \lambda \rangle$, dacă λ apare printre simbolurile din α , respectiv $\alpha \rangle \lambda \langle$ cazul contrar.

Definiția 1.9.2.1 Fie k clauză care nu este tautologie și $p \in V$. Clauza k este p -pozitivă, dacă $k \langle p \rangle$, p -negativă, dacă $k \langle (\neg p) \rangle$ respectiv este p -neutră, dacă $k \rangle (\neg p) \langle$ și $k \rangle p \langle$.

Pentru k clauză λ -pozitivă, notăm $k \setminus \lambda$ clauza rezultată prin eliminarea literalului λ din α .

Observație Fie $\lambda = (\neg p)$, $p \in V$. Convenim să acceptăm $(\neg \lambda)$ ca literal și anume $(\neg \lambda)$ reprezintă literalul propoziție elementară p . Justificarea acestei convenții rezidă din proprietatea $(\neg(\neg p)) \equiv p$.

Pentru k clauză și λ literal, notăm $k \setminus \lambda$ clauza rezultată prin eliminarea literalului λ din α .

Definiția 1.9.2.2 Fie $S(\alpha)$ o reprezentare clauzală liberă de tautologii și λ literal. Fie submulțimile de clauze

$$\begin{aligned}\alpha_\lambda^+ &= \{k \mid k \in S(\alpha), k \langle \lambda \rangle\} \\ \alpha_\lambda^- &= \{k \mid k \in S(\alpha), k \langle (\neg \lambda) \rangle\} \\ \alpha_\lambda^0 &= \{k \mid k \in S(\alpha), k \rangle \lambda \langle, k \rangle (\neg \lambda) \langle \} \\ POS_\lambda(\alpha) &= \alpha_\lambda^0 \cup \{k \setminus \lambda \mid k \in \alpha_\lambda^+\} \\ NEG_\lambda(\alpha) &= \alpha_\lambda^0 \cup \{k \setminus (\neg \lambda) \mid k \in \alpha_\lambda^-\}.\end{aligned}$$

Observație Evident, $\alpha_\lambda^+ = \alpha_{(\neg \lambda)}^-$, $\alpha_\lambda^- = \alpha_{(\neg \lambda)}^+$, $\alpha_\lambda^0 = \alpha_{(\neg \lambda)}^0$. Deoarece $S(\alpha)$ este reprezentare clauzală liberă de tautologii, pentru orice literal λ , α_λ^+ , α_λ^- , α_λ^0 este o partiție a mulțimii $S(\alpha)$.

De asemenea, $\alpha_\lambda^+ = \alpha_{(\neg \lambda)}^-$, $\alpha_\lambda^- = \alpha_{(\neg \lambda)}^+$, $\alpha_\lambda^0 = \alpha_{(\neg \lambda)}^0$.

Exemplul 1.9.2.1 Fie $S(\alpha) = \{k_1, k_2, k_3, k_4, k_5, k_6\}$, unde $k_1 = (\neg p) \vee o$, $k_2 = (\neg p) \vee (\neg c)$, $k_3 = (\neg m) \vee c \vee i$, $k_4 = m$, $k_5 = p$, $k_6 = (\neg i)$.

Pentru $\lambda = (\neg p)$,

$$\begin{aligned}\alpha_\lambda^+ &= \{(\neg p) \vee o, (\neg p) \vee (\neg c)\} \\ \alpha_\lambda^- &= \{p\} \\ \alpha_\lambda^0 &= \{(\neg m) \vee c \vee i, m, (\neg i)\} \\ POS_\lambda(\alpha) &= \{(\neg m) \vee c \vee i, m, (\neg i)\} \cup \{o, (\neg c)\} \\ NEG_\lambda(\alpha) &= \{(\neg m) \vee c \vee i, m, (\neg i)\} \cup \{\square\}.\end{aligned}$$

Observație Dacă $S(\alpha)$ este o reprezentare clauzală liberă de tautologii, atunci pentru orice literal λ , $POS_\lambda(\alpha)$, $NEG_\lambda(\alpha)$ sunt de asemenea reprezentări clauzale libere de tautologii. Literalul λ nu apare în clauzele reprezentărilor clauzale $POS_\lambda(\alpha)$, $NEG_\lambda(\alpha)$.

Definiția 1.9.2.3 Fie $S(\alpha)$ o reprezentare clauzală liberă de tautologii și λ literal. Spunem că λ este literal pur, dacă $\alpha_\lambda^- = \emptyset$. Clauza $k = \bigvee_{i=1}^n L_i$ este clauză unitară, dacă $n = 1$.

Teorema 1.9.2.1 (Teorema de separare Davis-Putnam). Fie $S(\alpha)$ o reprezentare clauzală liberă de tautologii. Dacă $S(\alpha)$ este validabilă, atunci pentru orice literal λ , cel puțin una dintre mulțimile de clauze $POS_\lambda(\alpha)$, $NEG_\lambda(\alpha)$ este validabilă. Dacă există un literal λ , astfel încât cel puțin una dintre mulțimile de clauze $POS_\lambda(\alpha)$, $NEG_\lambda(\alpha)$ este validabilă, atunci $S(\alpha)$ este validabilă.

Demonstrație Presupunem că $S(\alpha)$ este validabilă și fie $I \in \aleph(S(\alpha))$ arbitrară. Demonstrăm că pentru orice λ literal, dacă $I(\lambda) = T$, atunci

$I(NEG_\lambda(\alpha)) = T$, respectiv dacă $I(\lambda) = F$, atunci $I(POS_\lambda(\alpha)) = T$.

Presupunem $I(\lambda) = T$. Fie $k \in NEG_\lambda(\alpha)$ arbitrară.

a) dacă $k \in \alpha_\lambda^0$, cum $\alpha_\lambda^0 \subset S(\alpha)$ și $I(S(\alpha)) = T$, rezultă $I(k) = T$

b) dacă $k \in \{k^* \setminus (\neg\lambda) \mid k^* \in \alpha_\lambda^-\}$, fie $k^* \in \alpha_\lambda^-$, astfel încât $k = k^* \setminus (\neg\lambda)$, deci $k^* = k \vee (\neg\lambda)$.

Deoarece $k^* \in \alpha_\lambda^- \subset S(\alpha)$, rezultă $I(k^*) = T$, deci

$I(k^*) = I(k) \vee I((\neg\lambda)) = I(k)$.

Obținem astfel, $I \in \bigcap_{k \in NEG_\lambda(\alpha)} \aleph(k) = \aleph(NEG_\lambda(\alpha))$.

Un argument similar conduce la concluzia $I \in \aleph(POS_\lambda(\alpha))$ în cazul în care $I(\lambda) = F$.

Presupunem că există un literal λ , astfel încât cel puțin una dintre mulțimile de clauze $POS_\lambda(\alpha)$, $NEG_\lambda(\alpha)$ este validabilă.

Presupunem că $POS_\lambda(\alpha)$ este validabilă și fie $I \in \aleph(POS_\lambda(\alpha))$.

Definim funcția de adevăr $\bar{h} : V \rightarrow \{T, F\}$ astfel,

a) dacă $\lambda \in V$, atunci $\forall p \in V, \bar{h}(p) = \begin{cases} I(p), & \text{dacă } p \neq \lambda \\ F, & \text{dacă } p = \lambda \end{cases}$

b) dacă $\lambda \in \neg V$, atunci $\forall p \in V, \bar{h}(p) = \begin{cases} I(p), & \text{dacă } (\neg p) \neq \lambda \\ T, & \text{dacă } (\neg p) = \lambda \end{cases}$

Evident, rezultă $I(\bar{h})(\lambda) = F$ și pentru orice k clauză, astfel încât $k \rangle \lambda \langle$ și $k \rangle (\neg\lambda) \langle$, $I(\bar{h})(k) = I(k)$.

Fie $k \in S(\alpha)$ arbitrară.

a) dacă $k \in \alpha_\lambda^0$, atunci $I(\bar{h})(k) = I(k)$ și cum $\alpha_\lambda^0 \subset POS_\lambda(\alpha)$ rezultă $I(k) = T$, deci $I(\bar{h})(k) = T$;

b) dacă $k \in \alpha_\lambda^-$, atunci $k \equiv k_1 \vee (\neg\lambda)$, deci

$I(\bar{h})(k) = I(\bar{h})(k_1) \vee I(\bar{h})(\neg\lambda) = I(\bar{h})(k_1) \vee T = T$;

c) dacă $k \in \alpha_\lambda^+$, atunci $k \setminus \lambda \in POS_\lambda(\alpha)$, deci $I(k \setminus \lambda) = T$.

Deoarece $S(\alpha)$ este reprezentare clauzală liberă de tautologii $(k \setminus \lambda) \rangle \lambda \langle$ și $(k \setminus \lambda) \rangle (\neg\lambda) \langle$, deci $I(\bar{h})(k \setminus \lambda) = I(k \setminus \lambda)$.

Rezultă $I(\bar{h})(k \setminus \lambda) = T$. Evident, $k \equiv (k \setminus \lambda) \vee \lambda$, deci

$$I(\bar{h})(k) = I(\bar{h})(k \setminus \lambda) \vee I(\bar{h})(\lambda) = T \vee I(\bar{h})(\lambda) = T.$$

Obținem în final, $I(\bar{h}) \in \bigcap_{k \in S(\alpha)} \aleph(k) = \aleph(S(\alpha))$, deci $S(\alpha)$ este validabilă.

Un argument similar conduce la aceeași concluzie în ipoteza că $NEG_\lambda(\alpha)$ este validabilă.

Corolarul 1.9.2.1. Dacă $S(\alpha)$ este o reprezentare clauzală liberă de tautologii, atunci $\aleph(S(\alpha)) \subset \bigcap_{\lambda \in V \cup \neg V} (\aleph(NEG_\lambda(\alpha)) \cup \aleph(POS_\lambda(\alpha)))$.

Justificarea relației este imediată și rezultă din demonstrația *Teoremei 1.9.2.1.*

Corolarul 1.9.2.2. (Regula clauzei unitare) Fie $S(\alpha)$ reprezentare clauzală liberă de tautologii. Dacă λ este clauză unitară, atunci $S(\alpha)$ este validabilă, dacă și numai dacă $NEG_\lambda(\alpha)$ este validabilă.

Demonstrație Deoarece λ este clauză unitară, $\lambda \in \alpha_\lambda^+$. Rezultă

$\lambda \setminus \lambda = \square \in POS_\lambda(\alpha)$, deci cum clauza vidă este invalidabilă, rezultă $POS_\lambda(\alpha)$ invalidabilă. Dacă $S(\alpha)$ este validabilă, utilizând rezultatul stabilit de *Teorema 1.9.2.1.*, rezultă

$NEG_\lambda(\alpha)$ este validabilă. Reciproc, dacă $NEG_\lambda(\alpha)$ este validabilă, atunci din *Teorema 1.9.2.1* obținem că $S(\alpha)$ este validabilă.

Corolarul 1.9.2.3 (Regula literalilor puri) Fie $S(\alpha)$ reprezentare clauzală liberă de tautologii. Dacă λ este literal pur, atunci $S(\alpha)$ este validabilă, dacă și numai dacă $NEG_\lambda(\alpha)$ este validabilă.

Demonstrație Deoarece λ este literal pur, $\alpha_\lambda^- = \emptyset$ deci

$NEG_\lambda(\alpha) = \alpha_\lambda^0 \subset POS_\lambda(\alpha)$. Obținem

$$\aleph(POS_\lambda(\alpha)) = \bigcap_{k \in POS_\lambda(\alpha)} \aleph(k) \subset \bigcap_{k \in NEG_\lambda(\alpha)} \aleph(k) = \aleph(NEG_\lambda(\alpha)).$$

Dacă $S(\alpha)$ este validabilă și $I \in \mathfrak{S}$ este astfel încât $I(S(\alpha)) = T$, atunci din *Corolarul 1.9.2.1* obținem $I(POS_\lambda(\alpha)) = T$ sau

$$I(NEG_\lambda(\alpha)) = T.$$

Deoarece $\aleph(POS_\lambda(\alpha)) \subset \aleph(NEG_\lambda(\alpha))$, rezultă $I(NEG_\lambda(\alpha)) = T$, deci

$NEG_\lambda(\alpha)$ este validabilă.

Dacă $NEG_\lambda(\alpha)$ este validabilă, atunci din *Teorema 1.9.2.1* obținem direct concluzia $S(\alpha)$ validabilă.

Rezultatul stabilit de teorema de separare Davis-Putnam constituie suportul pentru dezvoltarea unei tehnici de căutare sistematică a unui model pentru o reprezentare clauzală dată.

Fie $S(\alpha)$ reprezentare clauzală liberă de tautologii și $\lambda_1, \dots, \lambda_n$ literalii care apar în clauzele din $S(\alpha)$. Fie reprezentările clauzale $\{\gamma_{i_1 \dots i_k} \mid i_1, \dots, i_k \in \{0, 1\}, 1 \leq k \leq n, \}$ rezultate astfel,

$$\begin{aligned} \gamma_0 &= NEG_{\lambda_1}(\alpha), \gamma_1 = POS_{\lambda_1}(\alpha), \gamma_{i_1 \dots i_{k-1} 0} = \\ &= NEG_{\lambda_k}(\gamma_{i_1 \dots i_{k-1}}), \gamma_{i_1 \dots i_{k-1} 1} = POS_{\lambda_k}(\gamma_{i_1 \dots i_{k-1}}), k = 2, \dots, n. \end{aligned}$$

Deoarece în clauzele reprezentării clauzale $\gamma_{i_1 \dots i_k}$ nu mai apar literalii $\lambda_1, \dots, \lambda_k$, obținem că toate reprezentările clauzale

$\gamma_{i_1 \dots i_n}, i_1, \dots, i_n \in \{0, 1\}$ conțin numai clauze fără literal. Rezultă că pentru orice

$$i_1, \dots, i_n \in \{0, 1\}, \gamma_{i_1 \dots i_n} \in \{\{\square\}, \emptyset\}.$$

Dacă $\gamma_{i_1 \dots i_n} = \{\square\}$ pentru toți $i_1, \dots, i_n \in \{0, 1\}$, atunci pe baza concluziei stabilite de *Teorema 1.9.2.1*, toate reprezentările clauzale $\gamma_{i_1 \dots i_{n-1}}$ rezultă invalidabile. Iterând același argument, obținem că toate reprezentările clauzale $\gamma_{i_1 \dots i_k}$, $i_1, \dots, i_k \in \{0, 1\}$, $1 \leq k \leq n$ sunt invalidabile, deci în final $S(\alpha)$ rezultă invalidabilă. Dacă există $i_1, \dots, i_n \in \{0, 1\}$, astfel încât $\gamma_{i_1 \dots i_n} = \emptyset$, atunci din *Teorema 1.9.2.1*, $\gamma_{i_1 \dots i_{n-1}}$ rezultă validabilă. Iterând același argument, obținem că toate reprezentările clauzale $\gamma_{i_1 \dots i_k}$, $1 \leq k \leq n$ sunt validabile, deci în final $S(\alpha)$ rezultă validabilă.

Metoda descrisă corespunde generării unui arbore binar complet cu vârfurile etichetate cu reprezentările clauzale $\gamma_{i_1 \dots i_k}$, $i_1, \dots, i_k \in \{0, 1\}$, $1 \leq k \leq n$ și rădăcina etichetată cu $S(\alpha)$. Orice vârf intermediar corespunde unei acțiuni de "separare" a reprezentării clauzale asociate ca etichetă a vârfului în "partea pozitivă (POS)" și "partea negativă (NEG)" în raport cu un anume literal. Utilizând rezultatele stabilite de *Corolarul 1.9.2.2* și *Corolarul 1.9.2.3*, obținem că, în cazul în care reprezentarea clauzală corespunzătoare unui vârf, fie conține o clauză unitară, fie există un literal pur, nu mai este necesară generarea ambilor fii ai vârfului, pentru vârful respectiv fiind suficientă generarea unui singur fiu. Într-adevăr, dacă reprezentarea clauzală γ corespunzătoare vârfului conține o clauză unitară λ , atunci $POS_\lambda(\gamma)$ este invalidabilă, deci toate vârfurile din subarborele de rădăcina etichetată prin reprezentarea clauzală $POS_\lambda(\gamma)$ vor avea ca etichete reprezentări clauzale invalidabile. Dacă în reprezentarea clauzală γ corespunzătoare vârfului există un literal pur λ atunci întreaga informație relativ la validabilitatea reprezentării γ este conținută de subarborele de rădăcina etichetată $NEG_\lambda(\gamma)$.

De asemenea, dacă reprezentarea clauzală, etichetă a unui vârf este formula vidă, atunci $S(\alpha)$ rezultă validabilă, deci nu mai este necesară extinderea în continuare a arborelui. Dacă reprezentarea clauzală etichetă a unui vârf conține clauza vidă, atunci toate reprezentările clauzale etichete ale vârfurilor din subarborele de rădăcină acel vârf vor fi invalidabile, deci devine inutilă extinderea arborelui în direcția acelui vârf.

Descrierea metodei de demonstrare automată rezultată pe baza acestor considerații este prezentată în continuare. Simularea generării arborelui etichetat cu reprezentări clauzale este realizată prin menținerea unei stive T în care sunt reținute vârfurile arborelui generate, dar încă neprelucrate.

```

procedure DvP;
Intrare: Structură de date pentru stocarea reprezentării clauzale  $S(\alpha)$ 
EliminaTautologii( $S(\alpha)$ );
 $\gamma \leftarrow S(\alpha)$ ;  $T \leftarrow \emptyset$ ;  $sw \leftarrow false$ ;
repeat
if  $\gamma = \emptyset$  then
    write ('validabilă');
     $sw \leftarrow true$ 
else
    if  $\square \in \gamma$  then
        if  $T = \emptyset$  then
            write ('invalidabilă');
             $sw \leftarrow true$ 
        else
             $\gamma \leftarrow TOP(T)$ ;
             $POP(T)$ ;
        endif
    else
        if (există  $\lambda$  clauza unitară) or
           (există  $\lambda$  literal pur)
        then
             $\gamma \leftarrow NEG_\lambda(\gamma)$ 
        else
            alege( $\lambda$  literal);
             $\gamma \leftarrow NEG_\lambda(\gamma)$ ;
             $PUSH(T, POS_\lambda(\gamma))$ 
        endif
    endif
endif
until sw
end

```

Observație Apelul $EliminaTautologii(S(\alpha))$ realizează detectarea și eliminarea clauzelor tautologii din reprezentarea clauzală $S(\alpha)$. Apelul este necesar doar la momentul inițial, deoarece dacă $S(\alpha)$ este o reprezentare clauzală liberă de tautologii, atunci toate reprezentările clauzale generate pe durata căutării inițiate de procedura DvP sunt reprezentări clauzale libere de tautologii. Apelul $alege(\lambda \text{ literal})$ efectuează selectarea unui literal pentru aplicarea operației de separare reprezentării clauzale curente.

Exemplul 1.9.2.2. Evoluția determinată de procedura DvP pentru datele

de intrare $S(\alpha) = \{k_1, k_2, k_3, k_4, k_5, k_6\}$, unde

$$\begin{aligned} k_1 &= (\neg p) \vee o, \quad k_2 = (\neg p) \vee (\neg c), \quad k_3 = (\neg m) \vee c \vee i, \\ k_4 &= m, \quad k_5 = p, \quad k_6 = (\neg i) \end{aligned}$$

este :

Inițializări: $\gamma \leftarrow \{(\neg p) \vee o, (\neg p) \vee (\neg c), (\neg m) \vee c \vee i, m, p, (\neg i)\}$;
 $sw \leftarrow false; T \leftarrow \emptyset$

Iterația 1: $\lambda = m$ clauză unitară

$$\gamma \leftarrow NEG_m(\gamma) = \{(\neg p) \vee o, (\neg p) \vee (\neg c), c \vee i, p, (\neg i)\}$$

Iterația 2 : $\lambda = p$ clauză unitară

$$\gamma \leftarrow NEG_p(\gamma) = \{o, (\neg c), c \vee i, (\neg i)\}$$

Iterația 3 : $\lambda = o$ clauză unitară (literalul o este și literal pur)

$$\gamma \leftarrow NEG_o(\gamma) = \{(\neg c), c \vee i, (\neg i)\}$$

Iterația 4 : $\lambda = (\neg c)$ clauză unitară

$$\gamma \leftarrow NEG_{(\neg c)}(\gamma) = \{i, (\neg i)\}$$

Iterația 5 : $\lambda = i$ clauză unitară

$$\gamma \leftarrow NEG_i(\gamma) = \{\square\}$$

Iterația 6 : $\square \in \gamma$ și $T = \emptyset \Rightarrow \text{write ('invalidabilă')}, sw \leftarrow true$
 $\Rightarrow \text{stop}$

Observație Deoarece algoritmul decide că reprezentarea clauzală $S(\alpha)$ este invalidabilă, rezultă $H \vdash \beta$, unde

$$\begin{aligned} \beta &= ((m \wedge p) \rightarrow i), \\ H &= \{\alpha_1 = (p \rightarrow (o \wedge (\neg c))), \alpha_2 = ((m \wedge (\neg c)) \rightarrow i)\} \end{aligned}$$

(*Exemplul 1.9.1.3*)

Exemplul 1.9.2.3 .Evoluția determinată de procedura DvP pentru datele de intrare $S(\alpha) = \{k_1, k_2, k_3, k_4\}$, unde

$$k_1 = a \vee (\neg b) \vee c, \quad k_2 = (\neg a) \vee (\neg c), \quad k_3 = (\neg c) \vee b, \quad k_4 = (\neg b) \vee a$$

este :

Inițializări: $\gamma \leftarrow \{a \vee (\neg b) \vee c, (\neg a) \vee (\neg c), (\neg c) \vee b, (\neg b) \vee a\}$;
 $sw \leftarrow false; T \leftarrow \emptyset$

Iterația 1: Nu există clauză unitară și nici literal pur;

apelul $\text{alege}(\lambda \text{ literal})$ selectează $\lambda = a$

$$\gamma \leftarrow NEG_a(\gamma) = \{(\neg c), (\neg c) \vee b\}$$

$$T \leftarrow POS_a(\gamma) = \{(\neg b) \vee c, (\neg c) \vee b, (\neg b)\}$$

Iterația 2 : $\lambda = (\neg c)$ clauză unitară (literalul $(\neg c)$ este și literal pur)

$$\gamma \leftarrow NEG_{(\neg c)}(\gamma) = \emptyset$$

Iterația 3 : $\gamma = \emptyset \Rightarrow$ decizia terminală 'validabilă'

5.3 1.9.3 Metoda bazată pe principiul rezoluției pentru verificarea validabilității formulelor limbajului calculului cu propoziții.

Rezoluția a fost introdusă ca regulă de inferență de către J.A.Robinson în 1965 și constituie sistemul deductiv cel mai frecvent utilizat în demonstrarea automată.

Reamintim că rezoluția este o regulă de inferență derivată, reprezentată convențional prin $\frac{(\alpha \rightarrow \beta), ((\neg \alpha) \rightarrow \gamma)}{(\beta \vee \gamma)} REZ$ (Aplicația 1.4.9).

Observație Deoarece $((\alpha \rightarrow \beta) \rightarrow (((\neg \alpha) \rightarrow \gamma) \rightarrow (\beta \vee \gamma))) \in T_h$, aplicând teorema deducției obținem $\{(\alpha \rightarrow \beta), ((\neg \alpha) \rightarrow \gamma)\} \vdash (\beta \vee \gamma)$ sau echivalent $\{(\alpha \rightarrow \beta), ((\neg \alpha) \rightarrow \gamma)\} \vdash \{(\beta \vee \gamma)\}$ și care din punct de vedere semantic revine la $\{(\alpha \rightarrow \beta), ((\neg \alpha) \rightarrow \gamma)\} \models \{(\beta \vee \gamma)\}$,

$$\text{adică } \aleph((\alpha \rightarrow \beta)) \cap \aleph(((\neg \alpha) \rightarrow \gamma)) \subset \aleph((\beta \vee \gamma)).$$

$$\text{Deoarece } \aleph((\alpha \rightarrow \beta)) = (\mathfrak{S} \setminus \aleph(\alpha)) \cup \aleph(\beta) = \aleph((\neg \alpha)) \cup \aleph(\beta) =$$

$$\aleph(((\neg \alpha) \vee \beta)) \text{ și}$$

$$\aleph(((\neg \alpha) \rightarrow \gamma)) = (\mathfrak{S} \setminus \aleph((\neg \alpha))) \cup \aleph(\gamma) = \aleph(\alpha) \cup \aleph(\gamma) = \aleph((\alpha \vee \gamma))$$

obținem,

$$\aleph(((\neg \alpha) \vee \beta)) \cap \aleph((\alpha \vee \gamma)) \subset \aleph((\beta \vee \gamma)),$$

$$\text{adică } \{((\neg \alpha) \vee \beta), (\alpha \vee \gamma)\} \models \{(\beta \vee \gamma)\},$$

$$\text{ceea ce este echivalent cu } \{((\neg \alpha) \vee \beta), (\alpha \vee \gamma)\} \vdash (\beta \vee \gamma).$$

Definiția 1.9.3.1 Fie k_1, k_2 clauze și λ literal. Clauzele k_1, k_2 sunt λ -rezolubile, dacă $k_1 \langle \lambda \rangle$ și $k_2 \langle (\neg \lambda) \rangle$. Clauza $rez_\lambda(k_1, k_2) = (k_1 \setminus \lambda) \vee (k_2 \setminus (\neg \lambda))$ este

λ -rezolventa perechii de clauze (k_1, k_2) . Clauzele k_1, k_2 se numesc clauze parentale ale rezolventei.

Observație În particular, pentru $\alpha = \lambda, \beta = (k_2 \setminus (\neg \lambda))$,

$\gamma = (k_1 \setminus \lambda)$, din observația precedentă rezultă $\{k_1, k_2\} \models rez_\lambda(k_1, k_2)$ sau echivalent, $\aleph(k_1) \cap \aleph(k_2) \subset \aleph(rez_\lambda(k_1, k_2))$. Evident, dacă nici una din clauzele k_1, k_2 nu este tautologie, atunci $rez_\lambda(k_1, k_2) \rangle \lambda \langle$ și $rez_\lambda(k_1, k_2) \rangle (\neg \lambda) \langle$.

Exemplul 1.9.3.1 Fie $k_1 = a \vee (\neg b) \vee c, k_2 = (\neg a) \vee (\neg c)$,

$$k_3 = (\neg b) \vee (\neg a).$$

Clauzele k_1, k_2 sunt a -rezolubile și c -rezolubile.

$$rez_a(k_1, k_2) = (\neg b) \vee c \vee (\neg c)$$

$$rez_c(k_1, k_2) = a \vee (\neg b) \vee (\neg a)$$

Clauzele k_1, k_3 sunt a -rezolubile

$$\text{rez}_a(k_1, k_3) = (\neg b) \vee c \vee (\neg b) \equiv c \vee (\neg b)$$

Clauzele k_2, k_3 nu sunt rezolubile

Din exemplele considerate rezultă că este posibil ca rezolventa a două clauze să fie clauză tautologie în condițiile în care ambele clauze parentale nu sunt tautologii. De asemenea, este posibil ca în rezolventa unei perechi de clauze să fie generate duplicate ale unuia sau mai mulți literal.

Definiția 1.9.3.2 Fie $S(\alpha)$ o reprezentare clauzală liberă de tautologii și λ literal. Rezoluția $REZ_\lambda(\alpha)$ în raport cu literalul λ este reprezentarea clauzală,

$$REZ_\lambda(\alpha) = \alpha_\lambda^0 \cup \{ \text{rez}_\lambda(k_1, k_2) \mid k_1 \in \alpha_\lambda^+, k_2 \in \alpha_\lambda^- \}$$

Exemplul 1.9.3.2 Dacă

$$S(\alpha) = \{ (\neg p) \vee o, (\neg p) \vee (\neg c), (\neg m) \vee c \vee i, m, p, (\neg i) \},$$

atunci,

$$REZ_p(\alpha) = \{ (\neg m) \vee c \vee i, m, (\neg i), o, (\neg c) \}$$

$$REZ_c(REZ_p(\alpha)) = \{ (\neg m) \vee i, m, (\neg i), o \}$$

$$REZ_i(REZ_c(REZ_p(\alpha))) = \{ (\neg m), m, o \}$$

$$REZ_m(REZ_i(REZ_c(REZ_p(\alpha)))) = \{ \square, o \}.$$

Teorema 1.9.3.1 (Teorema de bază a rezoluției). Fie $S(\alpha)$ o reprezentare clauzală.

Dacă $S(\alpha)$ este validabilă, atunci pentru orice literal λ , $REZ_\lambda(\alpha)$ este validabilă. Dacă există un literal λ , astfel încât $REZ_\lambda(\alpha)$ este validabilă, atunci $S(\alpha)$ este validabilă.

Demonstrație Presupunem că $S(\alpha)$ este validabilă și fie λ literal arbitrar. Fie $I \in \aleph(S(\alpha))$ arbitrară, deci $I \in \bigcap_{k \in S(\alpha)} \aleph(k)$.

Pentru orice $k \in REZ_\lambda(\alpha)$ obținem

a) dacă $k \in \alpha_\lambda^0$, deoarece $\alpha_\lambda^0 \subset S(\alpha)$ rezultă $I(k) = T$;

b) dacă $k = \text{rez}_\lambda(k_1, k_2)$ pentru anume $k_1 \in \alpha_\lambda^+, k_2 \in \alpha_\lambda^-$, atunci, cum $\{k_1, k_2\} \models \text{rez}_\lambda(k_1, k_2)$, rezultă $I(k) = T$.

Obținem astfel $I \in \bigcap_{k \in REZ_\lambda(\alpha)} \aleph(k) = \aleph(REZ_\lambda(\alpha))$, deci $REZ_\lambda(\alpha)$ este

validabilă. În particular, deoarece pentru orice $I \in \aleph(S(\alpha))$ a rezultat

$$I \in \aleph(REZ_\lambda(\alpha)), \text{ obținem } \aleph(S(\alpha)) \subset \aleph(REZ_\lambda(\alpha)).$$

Presupunem că există λ literal, astfel încât $REZ_\lambda(\alpha)$ este validabilă și fie $I \in \aleph(REZ_\lambda(\alpha))$ arbitrară.

Dacă $I \in \aleph(POS_\lambda(\alpha))$, atunci, utilizând rezultatul stabilit de

Teorema 1.9.2.1, rezultă $S(\alpha)$ validabilă. Dacă $I \notin \aleph(POS_\lambda(\alpha))$, atunci există $k \in POS_\lambda(\alpha)$, astfel încât $I(k) = F$. În acest caz, deoarece

$$\alpha_\lambda^0 \subset REZ_\lambda(\alpha), \text{ obținem } k = k_1 \setminus \lambda \text{ pentru anume } k_1 \in \alpha_\lambda^+.$$

Fie $k^* \in NEG_\lambda(\alpha)$ arbitrară.

a) dacă $k^* \in \alpha_\lambda^0 \subset REZ_\lambda(\alpha)$, atunci $I(k^*) = T$;
b) dacă $k^* = k_2 \setminus (\neg \lambda)$ pentru anume $k_2 \in \alpha_\lambda^-$, atunci k_1, k_2 sunt λ -rezolubile
și $rez_\lambda(k_1, k_2) \in REZ_\lambda(\alpha)$ deci $I(rez_\lambda(k_1, k_2)) = T$.

Obținem

$$\begin{aligned} I(rez_\lambda(k_1, k_2)) &= I((k_1 \setminus \lambda) \vee (k_2 \setminus (\neg \lambda))) = \\ &= I((k_1 \setminus \lambda)) \vee I((k_2 \setminus (\neg \lambda))) = I(k) \vee I(k^*) = F \vee I(k^*) = I(k^*), \end{aligned}$$

deci $I(k^*) = T$.

Rezultă $I \in \bigcap_{k^* \in NEG_\lambda(\alpha)} \aleph(k^*) = \aleph(NEG_\lambda(\alpha))$, deci $NEG_\lambda(\alpha)$ este validabilă. Utilizând rezultatul stabilit de *Teorema 1.9.2.1*, rezultă $S(\alpha)$ validabilă.

În particular, a rezultat că pentru orice $I \in \aleph(REZ_\lambda(\alpha))$, $I \in \aleph(POS_\lambda(\alpha))$ sau $I \in \aleph(NEG_\lambda(\alpha))$, deci

$$\aleph(REZ_\lambda(\alpha)) \subset \aleph(POS_\lambda(\alpha)) \cup \aleph(NEG_\lambda(\alpha)).$$

Corolarul 1.9.3.1 Dacă $S(\alpha)$ este o reprezentare clauzală, atunci

$$\begin{aligned} \aleph(S(\alpha)) &\subset \bigcap_{\lambda \in V \cup \neg V} \aleph(REZ_\lambda(\alpha)) \subset \\ &\subset \bigcap_{\lambda \in V \cup \neg V} (\aleph(NEG_\lambda(\alpha)) \cup \aleph(POS_\lambda(\alpha))). \end{aligned}$$

Demonstrație Din argumentele utilizate în demonstrația *Teoremei 1.9.3.1* a rezultat că pentru orice λ literal,

$$\aleph(S(\alpha)) \subset \aleph(REZ_\lambda(\alpha)), \text{ deci } \aleph(S(\alpha)) \subset \bigcap_{\lambda \in V \cup \neg V} \aleph(REZ_\lambda(\alpha)).$$

De asemenea, pentru fiecare λ literal, a rezultat relația

$$\aleph(REZ_\lambda(\alpha)) \subset \aleph(POS_\lambda(\alpha)) \cup \aleph(NEG_\lambda(\alpha)), \text{ deci}$$

$$\bigcap_{\lambda \in V \cup \neg V} \aleph(REZ_\lambda(\alpha)) \subset \bigcap_{\lambda \in V \cup \neg V} (\aleph(NEG_\lambda(\alpha)) \cup \aleph(POS_\lambda(\alpha))).$$

Definiția 1.9.3.3 Fie $S(\alpha)$ o reprezentare clauzală liberă de tautologii.

Secvența de clauze $k_1, \dots, k_n$ este o $S(\alpha)$ -derivare rezolutivă dacă pentru orice i , $1 \leq i \leq n$, este îndeplinită una din condițiile:

ι $k_i \in S(\alpha)$;

$\iota\iota$ există $1 \leq j, p \leq i$ și există λ literal, astfel încât $k_i = rez_\lambda(k_j, k_p)$.

$S(\alpha)$ -derivarea rezolutivă $k_1, \dots, k_n$ este o $S(\alpha)$ -respingere rezolutivă, dacă $k_n = \square$.

Teorema 1.9.3.2 (Teorema de completitudine a rezoluției). Fie $S(\alpha)$ o reprezentare clauzală. Atunci $S(\alpha)$ este invalidabilă, dacă și numai dacă

există o $S(\alpha)$ – respingere rezolutivă.

Demonstrație Demonstrăm că pentru orice $S(\alpha)$ – derivare rezolutivă

$k_1, \dots, k_n, S(\alpha) \models k_i, 1 \leq i \leq n$.

Evident, $k_1 \in S(\alpha)$, deci cum $\aleph(S(\alpha)) = \bigcap_{k \in S(\alpha)} \aleph(k) \subset \aleph(k_1)$, rezultă

$S(\alpha) \models k_1$.

Presupunem $S(\alpha) \models k_i, \forall i, 1 \leq i \leq j \leq n-1$.

Dacă $k_{j+1} \in S(\alpha)$, atunci din $\aleph(S(\alpha)) = \bigcap_{k \in S(\alpha)} \aleph(k) \subset \aleph(k_{j+1})$ obținem

$S(\alpha) \models k_1$.

Dacă există $i, p, 1 \leq i, p \leq j$ și λ literal, astfel încât

$k_{j+1} = rez_\lambda(k_i, k_p)$, utilizând ipoteza inductivă, obținem

$\aleph(S(\alpha)) \subset \aleph(k_i) \cap \aleph(k_p) \subset \aleph(rez_\lambda(k_i, k_p))$ deci $S(\alpha) \models k_{j+1}$.

Presupunem că există o $S(\alpha)$ – derivare rezolutivă $k_1, \dots, k_n = \square$.

Din $\aleph(S(\alpha)) \subset \aleph(\square) = \emptyset$ obținem evident $\aleph(S(\alpha)) = \emptyset$, deci $S(\alpha)$ este invalidabilă.

Reciproc, presupunem că $S(\alpha)$ este invalidabilă. Fără restrângerea generalității, putem presupune că $S(\alpha)$ este liberă de tautologii.

Fie $\lambda_1, \dots, \lambda_n$ literalii care au ocurențe în clauzele mulțimii $S(\alpha)$. Considerăm secvența de reprezentări clauzale $S_0(\alpha) = S(\alpha)$,

$S_i(\alpha) = REZ_{\lambda_i}(S_{i-1}(\alpha)), i = 1, \dots, n$.

Presupunem de asemenea că $S_i(\alpha), i = 1, \dots, n$ sunt libere de tautologii.

Rezultă că pentru fiecare $i = 1, \dots, n$, literalii $\lambda_1, \dots, \lambda_i$ nu au ocurențe în clauzele mulțimii $S_i(\alpha)$, deci $S_n(\alpha) = \emptyset$ sau

$S_n(\alpha) = \{\square\}$.

Dacă $S_n(\alpha) = \emptyset$, atunci $S_{n-1}(\alpha)$ este validabilă.

Din *Teorema* 1.9.3.1 rezultă $S_i(\alpha)$ validabilă, $i = 0, \dots, n$, deci $S(\alpha)$ este validabilă.

Analog, dacă $S_n(\alpha) = \{\square\}$, atunci $S_{n-1}(\alpha)$ este invalidabilă, deci iterând același argument, obținem $S_i(\alpha)$ invalidabilă, $i = 0, \dots, n$.

În ipoteza că $S(\alpha)$ este invalidabilă rezultă $S_n(\alpha) = \{\square\}$.

Dacă $S_i(\alpha) = \{\gamma_i^1, \dots, \gamma_i^{m_i}\}, i = 0, \dots, n$, unde $m_n = 1$ și $\gamma_n^1 = \square$, atunci $\gamma_1^1, \dots, \gamma_1^{m_1}, \gamma_2^1, \dots, \gamma_2^{m_2}, \dots, \gamma_{n-1}^1, \dots, \gamma_{n-1}^{m_{n-1}}, \gamma_n^1$ este o $S(\alpha)$ – derivare rezolutivă.

Deoarece $\gamma_n^1 = \square$, obținem în final că

$\gamma_1^1, \dots, \gamma_1^{m_1}, \gamma_2^1, \dots, \gamma_2^{m_2}, \dots, \gamma_{n-1}^1, \dots, \gamma_{n-1}^{m_{n-1}}, \gamma_n^1 = \square$

este o $S(\alpha)$ – respingere rezolutivă.

Rezultatul stabilit de *Teorema* 1.9.3.2 constituie suportul unui sistem de demonstrare automată pentru verificarea validabilității reprezentărilor clauzale. În esență, metoda revine la tentativa de obținere a unei respingeri rezolutive prin substituirea reprezentării clauzale curente printr-o reprezentare

clauzală cu număr de literal strict mai mic, dar posibil cu mai multe clauze. Deoarece prin luarea rezolventelor este posibilă generarea de clauze tautologii, în scopul menținerii reprezentărilor clauzale libere de tautologii este necesar ca la fiecare etapă să fie aplicat un test pentru detectarea și eliminarea rezolventelor tautologii nou generate. Spre deosebire de tehnica Davis-Putnam, implementarea demonstratorului rezolutiv nu necesită menținerea unei structuri de date suplimentare. Similar metodei Davis-Putnam, aplicarea priorității a regulilor literalilor puri și, respectiv, a clauzei unitare este o modalitate de prevenire a creșterii excesive a numărului de clauze.

Descrierea algoritmului de demonstrare automată rezolutiv este:

procedure DemRez;

Intrare: Structură de date pentru stocarea reprezentării clauzale $S(\alpha)$;

EliminaTautologii($S(\alpha)$);

EliminaDuplicate($S(\alpha)$);

$\gamma \leftarrow S(\alpha)$; $sw \leftarrow false$;

repeat

if $\gamma = \emptyset$ then

write ('validabilă');

$sw \leftarrow true$

else

if $\square \in \gamma$ then

write ('invalidabilă');

$sw \leftarrow true$

else

if (există λ clauza unitară) or

(există λ literal pur)

then

$\gamma \leftarrow NEG_{\lambda}(\gamma)$

else

alege(λ literal);

$\gamma \leftarrow RES_{\lambda}(\gamma)$;

EliminaTautologii(γ);

EliminaDuplicate(γ);

endif

endif

endif

until sw

end

Observație Similar algoritmului DvP, apelul EliminaTautologii(γ) detectează și elimină clauzele tautologii din reprezentarea clauzală curentă γ

(inițial $\gamma = S(\alpha)$). Apelul este necesar la fiecare etapă în care noua reprezentare clauzală este calculată ca rezoluție a reprezentării clauzale curente. Apelul EliminaDuplicate(γ) detectează și elimină duplicatele literalilor din fiecare clauză a reprezentării clauzale curente. Apelul alege(λ literal) efectuează selectarea unui literal pentru aplicarea rezoluției.

Exemplul 1.9.3.3 Procedura DemRez aplicată reprezentării clauzale

$$S(\alpha) = \{(\neg a) \vee (\neg c) \vee b, (\neg b) \vee c, (\neg c) \vee a, a \vee (\neg b) \vee (\neg c)\},$$

determină următoarea evoluție:

Inițializare: $\gamma \leftarrow S(\alpha)$, $sw \leftarrow false$.

Iterația 1: Nu există clauze unitare și nici literal puri.

Apelul alege(λ literal) $\Rightarrow \lambda = a$

$\gamma \leftarrow REZ_a(\gamma) = \{(\neg b) \vee c, (\neg c) \vee b \vee (\neg c), (\neg c) \vee b \vee (\neg b) \vee (\neg c)\}$

Apelul EliminaTautologii(γ) $\Rightarrow$ elimină clauza tautologie

$(\neg c) \vee b \vee (\neg b) \vee (\neg c)$

$\gamma \leftarrow \{(\neg b) \vee c, (\neg c) \vee b \vee (\neg c)\}$

Apelul EliminaDuplicate(γ) $\Rightarrow$ elimină duplicatul literalului $(\neg c)$ din $(\neg c) \vee b \vee (\neg c)$

$\gamma \leftarrow \{(\neg b) \vee c, (\neg c) \vee b\}$.

Iterația 2: Nu există clauze unitare și nici literal puri.

Apelul alege(λ literal) $\Rightarrow \lambda = b$

$\gamma \leftarrow REZ_b(\gamma) = \{c \vee (\neg c)\}$

Apelul EliminaTautologii(γ) $\Rightarrow$ elimină clauza tautologie $c \vee (\neg c)$

$\gamma \leftarrow \emptyset$

Apelul EliminaDuplicate(γ) $\Rightarrow$ nu determină modificări.

Iterația 3: $\gamma = \emptyset \Rightarrow$ write ('validabilă'), $sw \leftarrow true \Rightarrow$ stop.